DevSecret:

Local Encrypted CLI Vault for API Keys and Recovery Codes

Course Name: **CBM522** - Information Security and Applied Cryptography

Student Name: Meenakshi S L

Title

DevSecret: Local Encrypted CLI Vault for API Keys and Recovery Codes

Abstract

DevSecret is a Python command-line tool that stores developer API keys and two-factor authentication recovery codes in a single file-based vault on the local machine. Secrets are never sent to a cloud service: the vault is encrypted with AES-256-GCM after key derivation from a user-chosen master password using PBKDF2-HMAC-SHA256 (600,000 iterations). A separate plaintext sidecar index lists only site, username, and expiry metadata for API keys so that `list-keys` can run without decrypting key material. The project includes automated tests for cryptography, vault I/O, and CLI behavior, and is packaged as an installable application exposing the `devsecret` entry point.

Introduction

Developers routinely handle long-lived API keys and backup recovery codes. Storing them in plain text in shell history, dotfiles, or unencrypted notes increases exposure if a workstation is compromised or shared. Hosted secrets managers address enterprise needs but add account setup, network dependency, and operational overhead for individual or offline use.

DevSecret targets **local, offline-first** secret storage: one encrypted vault file, one master password, and a small set of subcommands to initialize the vault, add and retrieve API keys and recovery codes, list entries (with metadata-only listing for API keys via the index), and configure optional automatic removal of expired API keys. Expiry dates support hygiene for rotating keys, with warnings when a key is near or past its expiry. The design deliberately scopes risk: the index file is not encrypted and only reveals non-secret metadata; full confidentiality still depends on protecting the vault file and the master password.

Methodology

Requirements and stack. The implementation targets Python 3.10 or newer and depends on the `cryptography` library (version 42 or above) for PBKDF2, AES-GCM, and secure random salt/nonce generation.

Cryptographic design. Each save operation generates a random salt and a 12-byte GCM nonce, derives a 256-bit key from the UTF-8 master password with PBKDF2-HMAC-SHA256 (600k iterations), and encrypts the vault payload (UTF-8 JSON) with AES-256-GCM. The on-disk format includes a fixed magic string, a format version byte, salt, nonce, and ciphertext so that wrong passwords or tampering surface as decryption failures rather than silent corruption.

Data model and persistence. The vault JSON holds settings (including whether to auto-delete expired API keys), a nested map of API keys (per site and username, with optional expiry string), and recovery codes per site and username. Load normalizes structure; save writes encrypted bytes and refreshes the sidecar `vault.enc.index.json` with versioned index schema support for listing without decrypting secrets.

CLI workflow. `argparse` drives subcommands (`init`, `add-key`, `add-recovery`, `list-keys`, `list-recovery`, `get-key`, `get-recovery`, `configure`). The vault path resolves from `--vault`, then `DEVSECRET_VAULT`, then `~/.devsecret/vault.enc`. Sensitive input uses `getpass` where appropriate; API key retrieval prints the secret to stdout and routes expiry messages to stderr to ease shell scripting. Recovery codes can be supplied via flags, stdin, or interactive prompts.

Quality assurance. Tests under `tests/` exercise crypto edge cases, vault load/save and index behavior, and CLI integration using pytest fixtures and constants, supporting safe iteration on the vault format and commands.

Screenshots

add-key

```
● (venv) →  devsecret git:(main) ✗ devsecret add-key --site test.openai.com --username work
  Master password:
  API key:
  Warning: no --expiry given; using default expiry of 90 days (2026-07-20). Use --expiry DATE or --no-expiry to change this.
  Stored API key for test.openai.com / work
```

```
(venv) →  devsecret git:(main) ✗ devsecret add-key --site test.github.com --username alice --expiry 2026-12-31
Master password:
API key:
Stored API key for test.github.com / alice
```

```
(venv) →  devsecret git:(main) ✗ devsecret add-key --site internal.corp --username bot --no-expiry

Master password:
API key:
Stored API key for internal.corp / bot
```

add-recovery

```
● (.venv) →  devsecret git:(main) ✗ devsecret add-recovery --site test.com --username test --codes ABC-DEF,A123-sdfv
  Master password:
  Stored 2 recovery code(s) for test.com / test
○ (.venv) →  devsecret git:(main) ✗
```

list-keys

```
(.venv) → devsecret git:(main) ✗ devsecret list-keys
example.com     test
internal.corp   bot
test.github.com alice
test.openai.com work
(.venv) → devsecret git:(main) ✗
```

get-key

```
(.venv) → devsecret git:(main) ✗ devsecret get-key --site example.com --username test
Master password:
blahblahblah
```

list-recovery

```
(.venv) → devsecret git:(main) ✗ devsecret list-recovery
Master password:
github.com      meenakshisl
test.com        test
```

get-recovery

```
(.venv) → devsecret git:(main) ✗ devsecret get-recovery --site test.com --username test
Master password:
ABC-DEF
A123-sdfv
```

Code

The code has been implemented in python and has been committed to a public repository. It can be accessed here:

- https://github.com/meenakshisl/devsecret

Instructions are present in README.md

References

- *cryptography* (Python package), Fernet/AEAD and KDF usage - https://cryptography.io/
- NIST Special Publication 800-132, *Recommendation for Password-Based Key Derivation* (PBKDF2) - https://csrc.nist.gov/publications/detail/sp/800-132/rev-2/final
- NIST Special Publication 800-38D, *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM and GMAC)* - https://csrc.nist.gov/publications/detail/sp/800-38d/final

- OWASP *Cryptographic Storage Cheat Sheet* — https://cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.html